Сборник заданий для семинарских занятий
по курсу
«Объектно-ориентированное программирование на Python»

Содержание

1 Общие сведения **3**

2 Задания **3**

2.1 Семинар «Разработка простейшей программы с использованием ООП» (2 очных часа) 4

2.2 Семинар “Unit-тестирование” (2 очных часа) 8

2.2.1 Краткая теория 8

2.3 Семинары «SOLID» (8 очных часов) 19

2.4 Семинары «UML» (4 очных часа) 20

2.5 Семинары «Порождающие паттерны» (4 очных часа) 21

2.6 Семинары «Структурные паттерны» (6 очных часа) 22

1 Общие сведения

Сборник содержит задания для семинарских занятий по курсу «Объектно-ориентированное программирование на Python» (64 часа).

Курс предполагает повторение основ объектно-ориентированного программирования, освоение автоматического тестирования, овладение всеми основными паттернами объектно-ориентированного программирования и DDD, а также введение в паттерны асинхронного/параллельного программирования.

Возможна сдача другого кода (например, выполненного в ходе проектной деятельности), если они полностью покрывают материал семинара.

Оценивание работ осуществляется на основе следующих принципов:

- строгий учёт сроков сдачи (при просрочке учитываются официальные документы от врача и отправление на мероприятия в интересах ВУЗа, а также пропуск по причине назначения пересдач строго на время пары; возможен учёт других индивидуальных ситуаций), просрочка понижает оценку на 25 (но не ниже 50), базовая оценка – 100;
- каждая работа защищается (задаются вопросы и проводится собеседование по коду и общим принципам), если защита не успешна несколько раз, то оценка может быть понижена;
- если в работе обнаружены недостатки, то студенту предоставляется возможность их исправить без понижения оценки;
- оценка за практическую часть это средняя оценка всех работ с учетом того, что недосданные работы оцениваются пропорционально ниже оценки, которая была бы получена при условии сдачи;
- на пересдаче осуществляется сдача работ, оценка на пересдаче за конкретную работу не может быть выше 50 за исключением случая, когда имеются оправдательные документы согласно первому принципу.

Итоговое оценивание осуществляется независимо по теории и практике:

- по теории вычисляется средний балл для всех письменных опросов (пропущенные дополнительные вопросы по уважительной причине не учитываются; основные опросы учитываются всегда);
- по практике учитывается средний балл за все обычные практические задания и он имеет вес 40%, а также учитывается средний балл за все offline-практики (выполняемые на листочке по мотивам основных практических заданий) – этот балл имеет вес 60%;
- зачёт выставляется при достижении не менее 50 баллов по теории и не менее 50 баллов по практике;
- в случае необходимости выставления оценки в 5-балльной шкале шкала перевода следующая: от 84 баллов – 5, от 68 баллов – 4, от 50+ баллов – 3; на экзамене учитывается накопленный результат за год.

2 Задания

2.1 Семинар «Разработка простейшей программы с использованием ООП» (2 очных часа)

Осуществите разработку программы, основываясь на принципах ООП (инкапсуляция, наследование, полиморфизм, абстракция, повторное использование) в корректной трактовке. В необходимых случаях используйте `@staticmethod` и/или `@classmethod`, а таже модуль ABC.

Задание не предполагает оверинжиниринг (пока что принципы SOLID и паттерны ООП можно не использовать).

В этой и всех следующих работах соблюдайте следующие требования стиля кодирования:

- для удобства обращения к свойствам используйте `@property`;
- в целях возможности статического анализа кода используйте модуль typing и помечайте все аргументы функций типами, а также для функций указывайте тип возвращаемого значения;
- именование переменных, функций, классов должно осуществляться на английском языке (не транслит!), в целом используйте официальный стиль кодирования;
- на вход может подаваться любая информация, вам необходимо проверять корректность и самостоятельно выводить на экран информацию о проблемах.

Напишите программу, осуществляющую ввод информации с клавиатуры (в терминале) о сущностях, описанных в вашем варианте задания, и вывод на экран некоторых из них. Количество вводимых сущностей не ограничено. Проверяйте корректность входных данных.

1. Товары Интернет-магазина – книги и настольные игры. Все товары определяются ценой, книги имеют название, автора, количество страниц; настольные игры – название, число игроков. Выведите на экран все товары со стоимостью меньше 100 рублей.
2. Преподаватели определяются ФИО. Для тех, у кого есть диссертация, дополнительно вводится ее название; для остальных – стаж работы. Вывести всех преподавателей, у которых ФИО начинается на букву «А».
3. Наушники определяются названием модели. Беспроводные наушники дополнительно определяются протоколом передачи (Bluetooth, радиоканал) и дальностью действия; а проводные – типом разъёма (3.5 мм, USB-C) и наличием микрофона. Вывести все наушники, название которых начинается на «А».
4. Покатушки определяются названием и расстоянием. Однодневные покатушки дополнительно определяются плановым временем поездки (в часах). Многодневные покатушки определяются количеством дней и категорией сложности похода (от 1 до 6). Вывести все покатушки длиной более 100 км.
5. Музыкальная композиция определяется названием и композитором. Дополнительно для песни указывается автор стихов. Выведите информацию о всех композициях, у которых композитор начинается на букву «А».
6. Олимпиада определяется названием. Если олимпиада участвует в программе приема в ВУЗы дополнительно указывается уровень олимпиады (1–3), если олимпиада – этап всероссийской, то указывается название этапа (школьная, окружная, региональная, всероссийская), в остальных случаях – размер призового фонда. Выведите все олимпиады, название которых начинается на букву «А».

7. Проездной билет определяется стоимостью. Билет на количество поездок определяется количеством поездок. Билет на неограниченное количество поездок определяется сроком действия (1 день, 5 дней, 10 дней, 15 дней, месяц, три месяца, 6 месяцев, год). Выведите информацию о билетах, стоимостью меньше 300 рублей.

8. Информация о студенте определяется ФИО. Для студентов, не имеющих автомата, указывается балл, полученный на экзамене (2–5); для студентов, имеющих автомат указывается основание (олимпиада или контрольные работы). Если основание – контрольная работа, то дополнительно указывается средний балл за к/р. Выведите всю информацию о студентах с фамилией, начинающейся на буквы от А до К.

9. Сотовый телефон определяется названием. Для смартфонов указывается операционная система. А для других телефонов – наличие NFC. Выведите информацию о телефонах, название которых содержит слово «Xiaomi».

10. Куртка определяется названием модели, наличием капюшона. Для мембранных курток указывается степень водонепроницаемости (число в мм вод. ст.), для остальных – наличие пропитки. Выведите информацию обо всех куртках, имеющих капюшон.

11. Накопитель определяется названием и ёмкостью. Внешние накопители определяются дополнительно наличием системы защиты от падений, внутренние накопители – форм-фактором (M.2/2.5 дюйма) и типом (SSD или HDD). Выведите информацию о накопителях, ёмкость которых больше 500 Гб.

12. Велосипед определяется названием модели. Горному велосипеду соответствует количество скоростей, BMX – тип конструкции (фривил, кассетная, фрикостер). Выведите информацию обо всех велосипедах, содержащих в названии «Norco».

13. Электронная книга определяется названием и размером экрана. Для EInk-дисплея указывается поколение (Carta, Kaleido); для LCD – количество поддерживаемых цветов. Выведите информацию о всех книгах с размером экрана не менее 7 дюймов.

14. GPS определяется названием и диагональю экрана. Для переносных GPS указывается наличие велосипедного крепления; для автомобильных – поддержка отображения пробок и наличие радар-детектора. Выведите информацию обо всех GPS с размером экрана менее 7 дюймов.

15. Пылесос определяется названием модели. Для обычного пылесоса указывается мощность, для пылесоса-робота – размер убираемого помещения и количество виртуальных стен. Выведите информацию обо всех пылесосах, содержащих в названии слово Indesit.

16. Туры определяются названием. Для пляжного тура указывается тип пляжа (галечный, песок); для экскурсионного – количество экскурсий. Выведите информацию обо всех турах, содержащих слово Египет.

17. Язык программирования определяется названием. Алгоритмические языки определяются поддержкой ООП (отсутствует, на классах, прототипная), остальные языки – типом (функциональный, логический). Выведите информацию обо всех языках, название которых начинается с буквы «А».

18. Контрагенты определяются названием. Индивидуальные предприниматели дополнительно определяются наличием счета в банке, а юридические лица – организационно-правовой формой (ООО, АО, ПАО). Выведите информацию обо всех контрагентах, название которых начинается с буквы «А».

19. Счет в банке определяется номером. Для текущего счета указывается плата за обслуживание, для сберегательного счета – проценты годовых и наличие капитализации. Выведите информацию обо всех счетах, номер которых начинается с 408178...

20. Автомобильная дорога определяется названием и километражом. Бесплатная дорога определяется статусом автомагистрали (автомагистраль или нет), а платная – стоимостью за километр для обычных пользователей. Выведите информацию о дорогах, длина которых менее 100 км.

21. Офисное здание определяется адресом. В случае наличия стоянки указывается количество машиномест и стоимость аренды за месяц. Выведите информацию о зданиях, в адресе которых присутствует слово Тверская.

22. Товары Интернет-магазина – GPS-навигаторы и карты. Все товары определяются ценой и названием, GPS-навигаторы имеют назначение (ручной, автомобильный) и признак возможности загрузки карт; карты – размером (в Мб). Выведите информацию о всех товарах со стоимостью менее 4000 рублей.

23. Товары Интернет-магазина – чаи и кофе. Все товары определяются ценой, названием и весом, кофе – типом (растворимый, молотый, в зернах), чаи – типом (черный, зеленый). Выведите информацию о всех товарах с весом менее 150 г.

24. Объекты продаваемые в коттеджном поселке: участки (определяются площадью, стоимостью, наличием подряда), дома (определяются этажностью, площадью и стоимостью). Выведите все объекты со стоимостью меньше 1000000 рублей.

25. Вопросам теста соответствуют формулировка и количество баллов за правильный ответ. Вопросам с вариантами правильных ответов соответствуют 4 варианта ответа и номер правильного ответа; остальным вопросам – формулировка правильного ответа. Выведите все вопросы, оцениваемые в 10 баллов и выше.

26. Слова определяются собственно словом. Для существительных указывается род, для глаголов – спряжение. Выведите информацию обо всех словах, начинающихся на букву «А».

27. Операционная система определяется названием. Для операционной системы на базе Linux указывается название менеджера пакетов; для остальных – стоимость лицензии. Вывести все операционные системы, у которых название начинается на букву «А».

28. Рюкзаки определяются названием модели и емкостью. Для городских рюкзаков указывается наличие «вентилируемой спины», для походных – количество отделений и наличие крепления для трекинговых палок. Вывести информацию обо всех рюкзаках, в названии которых присутствует слово «Trek».

29. Автостоянка определяется названием, количеством машиномест. Для крытой автостоянки указывается количество этажей. Для открытой стоянки – наличие охраны. Вывести информацию обо всех автостоянках с количеством мест больше 20.

30. Партия определяется названием. Для тех партий, что финансируются из бюджета указывается размер ассигнований, а для остальных – количество депутатов в каких-либо представительных органах власти. Вывести информацию обо всех партиях с названиями, начинающимися на буквы от «А» до «К».

31. Маршрутизатор определяется названием и стоимостью. Для беспроводного маршрутизатора указывается стандарт Wi-Fi (Wi-Fi 4, Wi-Fi 5, Wi-Fi 6) и количество антенн, для проводного – количество портов. Выведите информацию обо всех маршрутизаторах, у которых количество портов или антенн больше 4.

32. Часы или фитнес-браслет определяется названием. Для спортивной модели указывается водозащита (в барах) и автономность (в часах), для детской – наличие GPS-трекинга и кнопки SOS. Выведите информацию обо всех моделях с автономностью более 24 часов.

33. Настольная игра определяется названием. Для кооперативных игр указывается число игроков и уровень сложности (от 1 до 5), для соревновательных – длительность партии (в минутах). Выведите информацию обо всех играх с длительностью партии более 60 минут.

34. Стриминговая подписка определяется названием сервиса и стоимостью (в месяц). Для музыкальной подписки указывается качество звука (потоковое, lossless) и наличие офлайн-режима, для видеосервисной – число одновременно используемых экранов и поддержка 4K. Выведите информацию обо всех подписках стоимостью менее 500 рублей в месяц.

35. Дрон определяется названием и ценой. Для любительского дрона указывается наличие камеры и дальность сигнала пульта (в метрах), для гоночного – максимальная скорость (в км/ч). Выведите информацию обо всех дронах стоимостью меньше 15000 рублей.

2.2 Семинар “Unit-тестирование” (2 очных часа)

Инструкция. Для своего варианта реализуйте указанную функцию и напишите набор unit-тестов, проверяющих все существенные случаи её работы.

2.2.1 Краткая теория

Unit-тест — автоматическая проверка отдельной функции на заранее известных входных данных и ожидаемом результате.

В Python для написания unit-тестов можно использовать встроенный модуль `unittest`.

Структура теста

Тесты обычно размещают в отдельном файле, имя которого начинается с `test_`:

```
from solution import classify_number
import unittest

class TestClassifyNumber(unittest.TestCase):

    def test_positive_even(self):
        self.assertEqual(
            classify_number(8),
            "positive even"
        )

if __name__ == "__main__":
    unittest.main()
```

Каждый метод, имя которого начинается с `test_`, является отдельным тестом.

Основные проверки

Наиболее часто используются следующие методы:

- `assertEqual(actual, expected)` — значения должны быть равны;
- `assertNotEqual(actual, expected)` — значения должны отличаться;
- `assertTrue(value)` — значение должно быть истинным;
- `assertFalse(value)` — значение должно быть ложным;
- `assertIn(value, collection)` — значение должно содержаться в коллекции;
- `assertRaises(Exception)` — ожидается возникновение указанного исключения. Используется как контекстный менеджер:

  ```
  with self.assertRaises(ZeroDivisionError):
      divide(1, 0)
  ```

Например:

```
self.assertEqual(is_perfect(6), True)
self.assertTrue(is_perfect(6))
self.assertFalse(is_perfect(10))
```

Что именно нужно проверять

Хороший набор тестов должен проверять не только обычный случай. Для функции следует определить различные классы входных данных и проверить каждый существенный случай.

Особенно важны:

- обычные корректные значения;
- граничные значения;
- значения непосредственно до и после границы;
- специальные случаи, например ноль или пустой список;
- некорректные входные данные, если они предусмотрены заданием;
- все ветви условных операторов `if`/`elif`/`else`.

Например, если функция использует условие

```
if x < 10:
...
elif x == 10:
...
else:
...
```

тесты должны обеспечить выполнение каждой из трёх ветвей.

Граничные значения

При наличии диапазонов необходимо проверять сами границы и значения по обе стороны от них.

Например, если скидка составляет 5% при $500 < price \leqslant 1000$, полезно проверить:

```
discount(500)
discount(501)
discount(1000)
discount(1001)
```

Такие тесты позволяют обнаружить ошибки в операторах `<`, `<=`, `>` и `>=`.

Запуск тестов

Отдельный тестовый файл можно запустить командой:

```
python -m unittest test_solution
```

Для автоматического поиска всех тестов:

```
python -m unittest discover
```

Если все тесты пройдены, программа сообщает об успешном выполнении. Если тест не пройден, необходимо сравнить фактический результат функции с ожидаемым и определить причину ошибки.

1. `classify_triangle(a, b, c)`

   Определяет тип треугольника по длинам трёх его сторон.

- если треугольник невозможно построить, вернуть `"invalid"`;
- если все три стороны равны, вернуть `"equilateral"`;
- если ровно две стороны равны, вернуть `"isosceles"`;
- если все три стороны различны, вернуть `"scalene"`.

Треугольник существует только при положительных длинах сторон и выполнении строгого неравенства треугольника: каждая сторона должна быть строго меньше суммы двух других. Вырожденный треугольник (где сумма двух сторон равна третьей) считается несуществующим и также возвращает `"invalid"`.

Стороны задаются целыми числами, поэтому сравнение суммы двух сторон с третьей выполняется точно, без погрешности.

2. `classify_number(n)`

   Определяет знак и чётность целого числа.

   Функция должна возвращать одну из строк:

   - `"positive even"` — положительное чётное число;
   - `"positive odd"` — положительное нечётное число;
   - `"negative even"` — отрицательное чётное число;
   - `"negative odd"` — отрицательное нечётное число;
   - `"zero"` — число равно нулю.

   Отдельно проверьте положительные и отрицательные числа каждой чётности, а также ноль.

3. `middle_value(a, b, c)`

   Возвращает среднее по величине из трёх чисел, то есть число, которое после упорядочивания трёх значений занимает вторую позицию.

   Функция не должна вычислять арифметическое среднее $(a + b + c)/3$.

   При наличии равных значений функция должна вернуть само значение, занимающее среднюю позицию. Проверьте случаи с тремя равными числами, с двумя равными числами и с тремя различными числами.

4. `max_of_three(a, b, c)`

   Возвращает наибольшее из трёх чисел. Если несколько чисел равны между собой и являются наибольшими, функция возвращает это значение.

   Реализация должна использовать явные ветвления `if`/`elif`/`else` без использования встроенных функций (например, `max`).

   Проверьте все варианты расположения наибольшего значения, а также случаи, когда два или все три числа равны.

5. `is_leap_year(year)`

   Определяет, является ли заданный год високосным.

   Год является високосным, если:

   - он делится на 400; или

- он делится на 4, но не делится на 100.

Во всех остальных случаях год невисокосный.

Обязательно проверьте обычный високосный год, обычный невисокосный год, год, кратный 100, но не кратный 400, и год, кратный 400.

6. `bmi_category(weight, height)`

   Вычисляет индекс массы тела:

   $$BMI = \frac{weight}{height^2},$$

   где `weight` задан в килограммах, а `height` — в метрах.

   Функция возвращает:

   - `"Underweight"` при $BMI < 18.5$;
   - `"Normal"` при $18.5 \leq BMI < 25$;
   - `"Overweight"` при $25 \leq BMI < 30$;
   - `"Obese"` при $BMI \geq 30$.

   Предполагается, что `weight` и `height` положительны. Поведение при `height` ≤ 0 или `weight` ≤ 0 не определено и в тестах не проверяется.

   Так как BMI — величина с плавающей точкой, при проверке границ допускается сравнение с точностью до 10^{-9}.

   Проверьте значения непосредственно на каждой границе категорий и по обе стороны от каждой границы.

7. `categorize_temperature(temp)`

   Определяет категорию температуры в градусах Цельсия. Функция работает с непрерывными числовыми значениями:

   - `"freezing"` при $temp \leq 0$;
   - `"cold"` при $0 < temp \leq 10$;
   - `"cool"` при $10 < temp \leq 20$;
   - `"warm"` при $20 < temp \leq 30$;
   - `"hot"` при $temp > 30$.

   Проверьте значения на границах и непосредственно по обе стороны от них.

8. `triangle_angle_type(a, b, c)`

   Определяет вид треугольника по длинам его сторон через соотношение между квадратами сторон.

   Функция возвращает:

   - `"invalid"`, если треугольник не существует;
   - `"right"`, если треугольник прямоугольный;
   - `"acute"`, если все его углы острые;

- `"obtuse"`, если один из углов тупой.

Для существующего треугольника сначала определите наибольшую сторону c. Затем сравнивайте c^2 с $a^2 + b^2$.

Стороны задаются целыми числами, поэтому все сравнения выполняются точно.

Проверьте также вырожденные и другие невозможные тройки сторон.

9. `quadrant(x, y)`

   Определяет положение точки (x, y) относительно координатных осей.

   Функция возвращает:

   - `1` — точка в первой четверти;
   - `2` — точка во второй четверти;
   - `3` — точка в третьей четверти;
   - `4` — точка в четвёртой четверти;
   - `"origin"` — точка $(0, 0)$;
   - `"axis"` — точка лежит на одной из координатных осей, но не является началом координат.

   Проверьте точки во всех четырёх четвертях, на обеих осях и в начале координат.

10. `days_in_month(month, leap)`

    Возвращает количество дней в указанном месяце.

    `month` — номер месяца от 1 до 12. `leap` — признак високосного года.

    Количество дней:

    - 31 день — январь, март, май, июль, август, октябрь, декабрь;
    - 30 дней — апрель, июнь, сентябрь, ноябрь;
    - февраль — 29 дней при `leap=True` и 28 дней при `leap=False`.

    Гарантируется, что `month` находится в диапазоне от 1 до 12. Поведение при других значениях `month` не определено и в тестах не проверяется.

    Проверьте каждый тип месяца и оба варианта февраля.

11. `traffic_fine(speed, zone)`

    Вычисляет размер штрафа за превышение скорости.

    Скорость в `speed` задаётся в км/ч. Для каждой зоны установлено следующее ограничение скорости:

    - `"residential"` — 40 км/ч;
    - `"city"` — 60 км/ч;
    - `"highway"` — 110 км/ч.

    Сначала вычисляется превышение:

    $$excess = speed - speed_limit.$$

    Размер штрафа:

- `"residential"`:
  - $excess \leq 10$ — 0;
  - $10 < excess \leq 20$ — 100;
  - $excess > 20$ — 200;
- `"city"`:
  - $excess \leq 15$ — 0;
  - $15 < excess \leq 30$ — 75;
  - $excess > 30$ — 150;
- `"highway"`:
  - $excess \leq 20$ — 0;
  - $20 < excess \leq 40$ — 50;
  - $excess > 40$ — 100.

Если `speed` меньше или равна ограничению, превышение отрицательно или равно нулю, и штраф равен 0. Если `speed` отрицательна, превышения нет, штраф равен 0.

Если зона не равна одной из трёх допустимых строк, функция возвращает `"invalid zone"`.

Проверьте отсутствие превышения, все границы штрафов и значения сразу по обе стороны каждой границы для каждой зоны.

12. `compare_three_numbers(a, b, c)`

    Сравнивает три числа и возвращает:

    - `"all equal"` — все три значения равны;
    - `"two equal"` — ровно два значения равны;
    - `"all different"` — все три значения различны.

    Проверьте все три возможных случая независимо от того, какая именно пара значений совпадает.

13. `max_digit(n)`

    Возвращает наибольшую цифру целого числа.

    Знак числа не учитывается: для отрицательного числа рассматривается абсолютное значение.

    Например, для $n = -582$ результатом является 8. Для числа 0 результат равен 0.

    Проверьте однозначные числа, многозначные числа, повторяющиеся цифры, а также положительные и отрицательные значения.

14. `triangle_area(a, b, c)`

    Вычисляет площадь треугольника по длинам его сторон, используя формулу Герона.

    Если треугольник не существует (включая вырожденный случай, когда сумма двух сторон равна третьей), функция возвращает `"invalid"`.

    Стороны задаются целыми числами. Площадь возвращается как число с плавающей точкой; при проверке в тестах допускается сравнение с точностью до 10^{-6}.

    Проверьте корректные треугольники, вырожденные случаи и заведомо невозможные тройки сторон.

15. `next_day(day, month, leap)`

    Возвращает дату следующего календарного дня.

    `day` — номер дня месяца, `month` — номер месяца, `leap` — признак високосного года.

    В обычный день увеличивается только номер дня. Если текущий день является последним днём месяца, результатом является первый день следующего месяца. После 31 декабря результатом является 1 января.

    Количество дней в феврале зависит от значения `leap`.

    Гарантируется, что входная дата корректна: `month` от 1 до 12, `day` от 1 до количества дней в этом месяце с учётом `leap`.

    Проверьте обычный переход внутри месяца, переход на следующий месяц, переход с февраля, переход с 31 декабря и оба варианта високосности февраля.

16. `is_inside_rectangle(x, y, x1, y1, x2, y2)`

    Проверяет, принадлежит ли точка (x, y) прямоугольнику, заданному двумя противоположными вершинами $(x1, y1)$ и $(x2, y2)$.

    Граница прямоугольника считается частью прямоугольника.

    Функция возвращает `True`, если точка находится внутри прямоугольника или на его границе, и `False` в противном случае.

    Координаты противоположных вершин могут быть заданы в любом порядке.

17. `discount(price)`

    Вычисляет итоговую цену товара после применения скидки.

    Правила:

    - при $price > 1000$ предоставляется скидка 10%;
    - при $500 < price \leq 1000$ предоставляется скидка 5%;
    - при $price \leq 500$ скидка не предоставляется.

    Функция возвращает именно цену после скидки, а не размер скидки.

    Предполагается, что `price` неотрицательна. Поведение при отрицательной цене не определено и в тестах не проверяется.

    Так как `price` может быть числом с плавающей точкой, при проверке результата допускается сравнение с точностью до 10^{-9}.

    Проверьте значения на обеих границах и непосредственно по обе стороны от них.

18. `closest_to_zero(lst)`

    Возвращает элемент списка, абсолютное значение которого минимально.

    Если два или более элемента находятся на одинаковом расстоянии от нуля, возвращается первый из них в исходном списке.

    Список содержит хотя бы один элемент.

    Проверьте положительные и отрицательные значения, наличие нуля, а также случай равного расстояния до нуля.

19. `season(month)`

    По номеру месяца возвращает название соответствующего времени года:

- `"Winter"` — декабрь, январь, февраль;
- `"Spring"` — март, апрель, май;
- `"Summer"` — июнь, июль, август;
- `"Autumn"` — сентябрь, октябрь, ноябрь.

Гарантируется, что `month` находится в диапазоне от 1 до 12. Поведение при других значениях не определено и в тестах не проверяется.

Проверьте все месяцы, включая месяцы, являющиеся границами сезонов.

20. `simple_calculator(a, b, op)`

    Выполняет одну арифметическую операцию над числами a и b.

    Параметр `op` определяет операцию:

    - `+` — $a + b$;
    - `-` — $a - b$;
    - `*` — $a \cdot b$;
    - `/` — a / b.

    При попытке деления на ноль функция возвращает `"error"`. При неизвестном операторе функция также возвращает `"error"`.

    Проверьте каждую операцию, деление на ноль и недопустимый оператор.

21. `sum_positive(lst)`

    Возвращает сумму всех строго положительных элементов списка.

    Ноль и отрицательные элементы не учитываются.

    Если в списке нет положительных элементов, результатом является 0.

    Проверьте список с положительными и отрицательными числами, список без положительных чисел и список, содержащий нули.

22. `sum_even(lst)`

    Возвращает сумму всех чётных элементов списка.

    Чётность определяется для целых чисел; отрицательные чётные числа также учитываются.

    Если чётных элементов нет, функция возвращает 0.

    Проверьте положительные, отрицательные, нулевые и нечётные элементы.

23. `sum_odd(lst)`

    Возвращает сумму всех нечётных элементов списка.

    В сумму включаются как положительные, так и отрицательные нечётные числа. Ноль не является нечётным и в сумму не включается.

    Если нечётных элементов нет, функция возвращает 0.

    Проверьте смешанный список, список без нечётных чисел и наличие отрицательных значений.

24. `reverse_signs(lst)`

    Возвращает новый список, в котором каждый элемент исходного списка заменён на противоположный по знаку:

    $$x \mapsto -x.$$

    Ноль остаётся равным нулю.

    Исходный список не должен изменяться.

    Проверьте положительные, отрицательные и нулевые элементы, а также пустой список.

25. `nearest_multiple(n, m)`

    Возвращает целое число, кратное m, которое находится на минимальном расстоянии от n.

    Параметры: n — целое число, m — целое положительное число ($m > 0$). Множество возможных результатов имеет вид $k \cdot m$, где k — целое число.

    Если два кратных находятся от n на одинаковом расстоянии, возвращается меньшее из них.

    Проверьте случаи, когда n уже кратно m, находится между двумя кратными, а также случай равного расстояния.

26. `sort_three(a, b, c)`

    Возвращает тройку чисел в порядке неубывания: от наименьшего значения к наибольшему.

    Например, для входа $(3, 1, 2)$ результатом является $(1, 2, 3)$.

    Реализация должна использовать ветвления `if`/`elif`/`else`.

    Повторяющиеся значения должны сохраняться в результате. Проверьте все варианты порядка трёх различных чисел и случаи с повторениями.

27. `validate_password(password)`

    Проверяет, соответствует ли строка пароля всем трём требованиям:

    - длина строки не менее 8 символов;
    - строка содержит хотя бы одну цифру от `0` до `9`;
    - строка содержит хотя бы одну заглавную латинскую букву от `A` до `Z`.

    Функция возвращает `True`, если выполнены все три условия, и `False` в противном случае.

    Проверьте корректный пароль, а также случаи нарушения каждого требования по отдельности и нескольких требований одновременно.

28. `is_perfect(n)`

    Проверяет, является ли целое положительное число совершенным.

    Число называется совершенным, если оно равно сумме всех своих положительных собственных делителей, то есть делителей, меньших самого числа.

    Например, $6 = 1 + 2 + 3$, поэтому 6 является совершенным числом.

Для $n \leq 0$ функция возвращает `False`.

Проверьте известное совершенное число, обычное несовершенное число, простое число и значения, меньшие или равные нулю.

29. `sum_digits(n)`

    Возвращает сумму всех цифр целого числа.

    Знак числа не учитывается: для отрицательного числа рассматривается абсолютное значение.

    Например, для -305 результатом является $3 + 0 + 5 = 8$.

    Для 0 результат равен 0.

    Проверьте однозначные и многозначные числа, наличие нулей внутри числа и отрицательные значения.

30. `count_vowels(s)`

    Возвращает количество английских гласных букв в строке.

    Гласными считаются символы: `a`, `e`, `i`, `o`, `u` в любом регистре.

    Все остальные символы, включая согласные, цифры, пробелы и знаки препинания, не учитываются.

    Проверьте строку без гласных, строку только из гласных, смешанный текст, разный регистр и пустую строку.

31. `is_palindrome(s)`

    Проверяет, является ли строка палиндромом.

    Палиндромом считается строка, которая совпадает со своей записью в обратном порядке.

    Сравнение выполняется посимвольно с учётом регистра и пробелов. Например, `"level"` является палиндромом, а `"Level"` — нет.

    Пустая строка и строка из одного символа считаются палиндромами.

    Проверьте палиндромы чётной и нечётной длины, непалиндромные строки, строку из одного символа и пустую строку.

32. `remove_duplicates(lst)`

    Возвращает новый список, содержащий каждый встречающийся элемент ровно один раз.

    Порядок элементов должен сохраняться: при повторном появлении элемента остаётся его первое вхождение.

    Например,

    $$[3, 1, 3, 2, 1] \to [3, 1, 2].$$

    Исходный список не должен изменяться.

    Проверьте список без повторов, список со множественными повторами, список из одинаковых элементов и пустой список.

33. `factorial_iterative(n)`

    Вычисляет факториал неотрицательного целого числа:

    $$n! = 1 \cdot 2 \cdot 3 \cdots n.$$

    По определению,

    $$0! = 1.$$

    Функция должна использовать цикл и не должна использовать рекурсию.

    Параметр n — целое число. Для отрицательного n функция должна возвращать `"error"`.

    Проверьте 0, 1, несколько обычных значений и отрицательное число.

34. `fizz_buzz(n)`

    Возвращает список результатов для всех целых чисел от 1 до n включительно.

    Правила замены:

    - число, кратное 3, заменяется на `"Fizz"`;
    - число, кратное 5, заменяется на `"Buzz"`;
    - число, кратное одновременно 3 и 5, заменяется на `"FizzBuzz"`;
    - остальные числа остаются числами.

    Проверка кратности 15 должна иметь приоритет над отдельной проверкой кратности 3 или 5.

    Для $n \leq 0$ функция возвращает пустой список.

    Проверьте значения до первой замены, значения, кратные только 3, только 5 и одновременно 3 и 5, а также границу диапазона.

35. `is_prime(n)`

    Проверяет, является ли целое число простым.

    Простым считается натуральное число, имеющее ровно два положительных делителя: 1 и само число.

    Числа меньше 2 простыми не являются.

    Параметр n — целое число.

    Функция возвращает `True` для простого числа и `False` для составного или недопустимого числа.

    Проверьте числа меньше 2, число 2, нечётные простые числа, чётные составные числа, составные нечётные числа и несколько чисел с делителями.

2.3 Семинары «SOLID» (8 очных часов)

В течение четырёх пар практических занятий предполагается внедрить принципы SOLID в код, полученный в первой практической работе, одновременно улучшив отзывчивость консольного интерфейса (CLI).

В этой работе (и далее в курсе) мы будем придерживаться очень строгого (и даже оверинжинирингового) подхода к реализации SOLID: будем выполнять принцип S максимально строго, даже если в реальной практике так делать не стоит.

Порядок вашей работы:

- реализовать отзывчивый CLI (то есть сделать максимально удобными ввод и вывод значений во всех ситуациях);
- реализовать принцип S так, чтобы каждый класс имел строго одну ответственность в максимально строгом смысле (по мере демонстрации работ преподавателю вы будете получать указания, что ещё следует выделить);
- реализовать принцип D так, чтобы почти каждый класс реализовывал интерфейс (мы будем создавать абстрактные классы без свойств и только с абстрактными методами и использовать множественное наследование); если необходимо использовать объекты других классов, передавайте их в конструктор или метод не как конкретные типы, а как типы интерфейсов;
- реализовать принцип I так, чтобы в конструкторы и методы передавались максимально узкие интерфейсы;
- реализовать принцип O так, чтобы класс был открыт для расширения, но закрыт для изменения;
- считать, что принцип L реализуется автоматически, если вы следуете правилам наследования, изложенным на первом занятии;
- реализовать автоматизированное модульное (unit) тестирование всех полученных классов (для этого понадобится создать mock-объекты или mock-классы);
- добавить документацию (docstring).

Документация пишется для каждого класса: краткое описание (однострочное), для каждого метода: краткое описание, аргументы, результаты, возможные исключения; необходимо отмечать подразумеваемые вещи (требования к аргументам и контексту, которые подразмеваются, но не проверяются).

2.4 Семинары «UML» (4 очных часа)

Разработайте документ в текстовом процессоре, в котором полноценно описывается результат, полученный в предыдущей практической работе, с точки зрения объектно-ориентированной архитектуры. Утверждения сопровождайте UML-диаграммами, которые:

- должны пояснять ваш текст;
- не должны быть перегруженными (в каждой диаграмме присутствуют только те классы и методы, которые поясняют описываемый аспект);
- должны строго соответствовать спецификации UML 2.5.1 (обращайте внимание на корректное использование стрелок).

В работе предполагается, что вы будете использовать:

- диаграммы классов;
- диаграммы последовательности.

2.5 Семинары «Порождающие паттерны» (4 очных часа)

- Используя паттерны «Фабричный метод» и «Абстрактная фабрика» реализуйте абстрактное создание и работу с различными сущностями в вашей программе (в каждом варианте по две опции сущностей). По результатам использования паттернов у вас должно упроститься добавление новых видов сущностей
- Используя паттерны «строитель» и «команда» (этот паттерн мы используем заранее) реализуйте создание меню из любого количества пунктов с любыми выполняемыми действиями
- Реализуйте запись и чтение из файла вашей базы данных (формат JSON) в следующей логике: после открытия программы пользователь выбирает: создать новую БД (и имя файла), открыть старую БД (и имя файла), работать в старом режиме (без файла). Файл открывается только тогда, когда он первый раз нужен и дальше не закрывается, пока пользователь явно не выберет операцию закрытия файла. Здесь используется паттерн «ленивая инициализация».
- Добавить возможность клонировать сущность (создается копия и можно отредактировать поля по выбору: проще дать возможность редактировать все поля, и если пользователь просто нажмет Enter, то использовать старое значение). Здесь мы используем паттерн «Прототип».

2.6 Семинары «Структурные паттерны» (6 очных часа)

Обеспечьте то, что в вашей программе можно будет добавлять, удалять и изменять сущности (выбирая соответствующие пункты меню; выбирать изменяемую или удаляемую сущность можно любым простым способом, например, по порядковому номеру при выводе на экран).

- Используя паттерны «Адаптер» и «Мост» реализуйте два варианта консольного интерфейса: тот, что был раньше и цветной (библиотека rich). Выбор осуществляется передачей параметра командной строки (`--colored` – значит новый вариант);
- Реализуйте возможность объединять сущности, что уже были раньше, в папки (папки могут быть внутри папок). Для этого используется паттерн «Компоновщик». Достаточно обеспечить возможность создавать новые папки и при создании сущности или папки выбирать ту, в которой она будет находиться;
- Реализуйте возможность протоколировать операции, что происходят внутри хранилища по выбору пользователя (для этого используется паттерн «Декоратор»). Реализуйте две опции `--log` *имя файла* и `--log-json` *имя файла*, которые можно использовать независимо, и осуществляющие протоколирование в обычном текстовом формате и в формате json соответственно
- Создайте класс, инкапсулирующий всю работу вашей системы, кроме ввода-вывода, чтобы можно было любые необходимые действия осуществлять посредством вызова методов этого класса (паттерн «фасад»)
- Дайте возможность указывать, что сущность в вашей программе (или папка), не доступны для изменения или удаления; контролируйте это с помощью паттерна «заместитель»
- Добавьте возможность выбирать цвет фона для отображения каждой сущности и папки, создаваемой пока цвет фона актуален. Актуальность меняется, когда пользователь выбирает новый цвет фона. Здесь используется паттерн «Приспособленец»